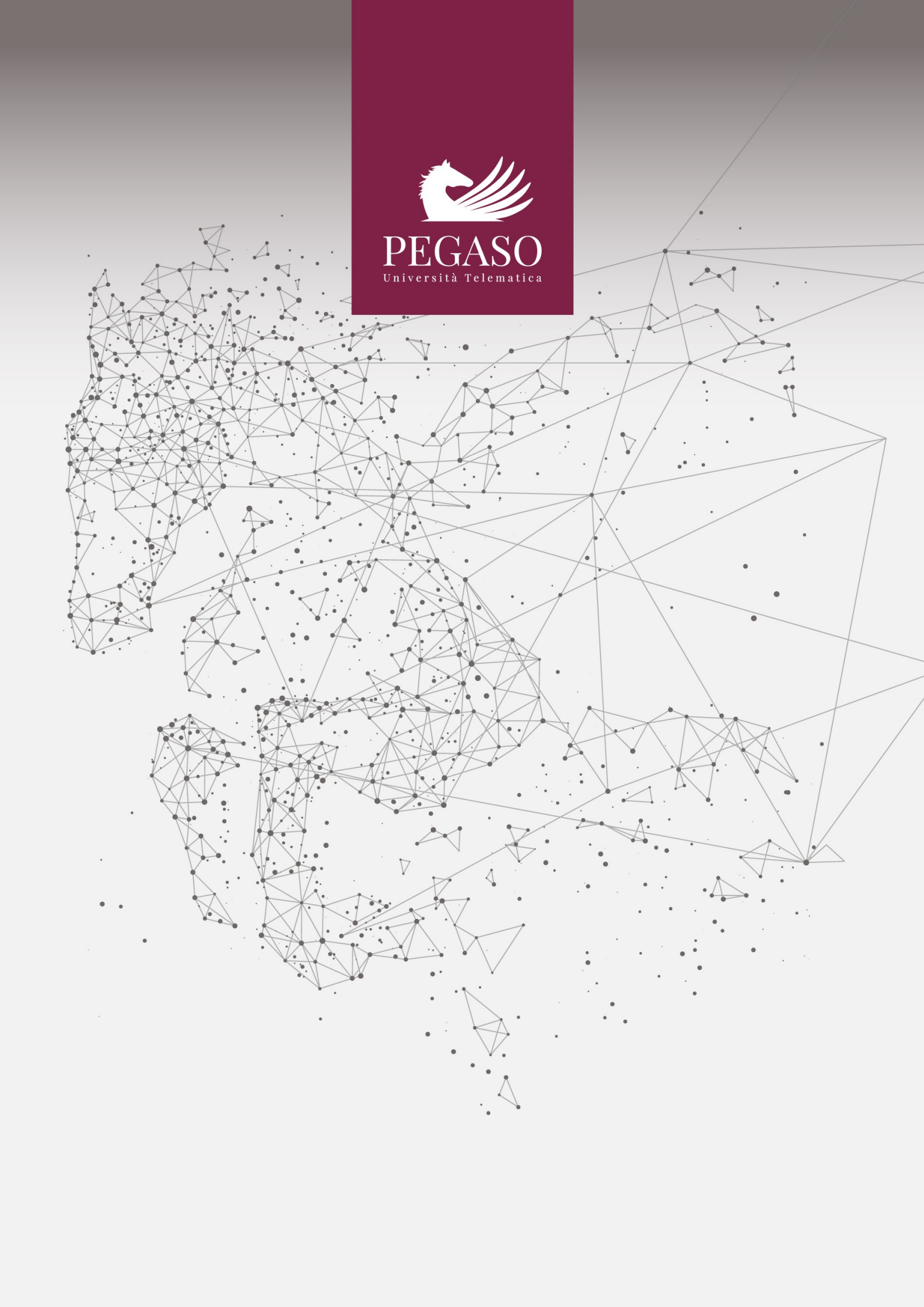

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. GESTIONE DEI DATI MANCANTI 4
3. RIPRODUCIBILITÀ NELL'ANALISI DEI BIG DATA 9
4. RACCOLTA DEI DATI IN MODO RIPRODUCIBILE 12
5. RIPRODUCIBILITÀ DEL CODICE 18
BIBLIOGRAFIA 20

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano due sfide fondamentali dell'analisi dei dati: la gestione dei dati mancanti e la riproducibilità computazionale. La prima permette di rispondere ad un problema molto frequente del mondo reale, quello dei dati mancanti nei dataset principalmente per motivi quali la mancata risposta del sistema e il danneggiamento e/o cancellazione parziale dei dati. La seconda permette di aumentare la facilità di riproduzione del flusso di lavoro e del codice. A tal proposito, vengono presentate le funzionalità offerte da Jupyter Notebook e dalle librerie Python e viene illustrato un esempio pratico di applicazione di tali funzionalità.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Gestione dei dati mancanti

I dati delle osservazioni contenute in un dataset che non hanno valori assegnati per una o più variabili sono definiti **valori mancanti.** Nel mondo reale è ***molto frequente*** il fenomeno dei valori mancanti nei dati. I valori possono essere assenti per diversi motivi come ad esempio:

- *mancata risposta del sistema;*
- *dati danneggiati;*
- *cancellazione parziale.*

Alcuni campi hanno *maggiori probabilità di contenere valori mancanti* rispetto ad altri. Ad esempio, è probabile che i **dati sul reddito** raccolti nei sondaggi contendano valori mancanti perché le persone non amano rivelare i loro guadagni.

Ad ogni modo, i *valori mancanti* rappresentano uno dei **maggiori problemi del mondo dell'analisi dei dati.** In base alla **percentuale di dati mancanti**, i valori mancanti possono rivelarsi una *sfida significativa* nella **preparazione dei dati** e **nell'analisi preliminare.** È, quindi, di fondamentale importanza calcolare la *percentuale di dati mancanti prima di iniziare l'analisi dei dati.*

Per fornire un esempio della **gestione dei dati mancanti** in un dataset, partiremo dal dataset open source ***Iris*** che fa parte della *libreria scikit-learn.* Il ***dataset Iris*** è un dataset con più variabili contenente *150 record*. Più in dettaglio, *50 record* per ognuna delle *3 specie di fiore Iris*:

1. *Iris Setosa;*
2. *Iris Virginica;*
3. *Iris Versicolor.*

Il dataset Iris è un dataset con **5 attributi/colonne** per ogni specie di Iris:

1. *petal length;*
2. *petal width;*
3. *sepal length;*
4. *sepal width;*
5. *species.*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il dataset è stato salvato in un **file CSV** esterno che sarà caricato in un dataframe Spark attraverso il seguente codice:

```
import findspark
findspark.init()
import pyspark
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

df = spark.read.csv('iris.csv', inferSchema = True, header = True)
type(df)
df.show(4)
```

```
+-----------+----------+-----------+----------+-------+
|sepallength|sepalwidth|petallength|petalwidth|variety|
+-----------+----------+-----------+----------+-------+
|        5.1|       3.5|        1.4|       0.2| Setosa|
|        4.9|       3.0|        1.4|       0.2| Setosa|
|        4.7|       3.2|        1.3|       0.2| Setosa|
|        4.6|       3.1|        1.5|       0.2| Setosa|
+-----------+----------+-----------+----------+-------+
only showing top 4 rows
```

Il dataset Iris *non ha valori mancanti*, quindi sono stati **rimossi casualmente** due valori dalla colonna '***sepallength***' (ovviamente di due righe diverse) e un valore dalla colonna '***petallength***' (in una terza riga) del dataset originale. Il dataset, quindi, contiene ora valori mancanti (file *iris_dati_mancanti.csv*). Nei programmi che seguiranno è fondamentale importare anche altre librerie come riportato nel codice seguente:

```
from pyspark.sql.functions import isnan, when, count, col
```

Per prima cosa conviene contare i **valori che mancano in una colonna** (es. '*sepallength*') del dataframe di PySpark attraverso il seguente codice:

```
dfm.filter(col('sepallength').isNull()).count()
```

```
2
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Oppure è possibile contare i valori che mancano per ciascuna colonna del dataframe di PySpark attraverso il seguente codice:

```
dfm = spark.read.csv('iris_dati_mancanti.csv', inferSchema = True, header = True)

dfm.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in dfm.columns]).show()
```

```
+-----------+----------+-----------+----------+-------+
|sepallength|sepalwidth|petallength|petalwidth|variety|
+-----------+----------+-----------+----------+-------+
|          2|         0|          1|         0|      0|
+-----------+----------+-----------+----------+-------+
```

Ecco che si ritrovano i tre valori mancanti che sono stati appositamente cancellati in precedenza. Attraverso il seguente codice è possibile, poi, stabilire una **percentuale dei valori mancanti** all'interno del dataset:

```
l = []
for i in dfm.columns:
    l.append(dfm.filter(col(i).isNull()).count())
nm = sum(l)
nt = dfm.count()*len(l)
print("Elementi mancanti:",nm)
print("Elementi totali:",nt)
print("Percentuale elementi mancanti:",nm/nt*100,"%")
```

```
Elementi mancanti: 3
Elementi totali: 750
Percentuale elementi mancanti: 0.4 %
```

Un modo alternativo e più semplice per individuare i dati mancanti è quello di richiamare il metodo **describe**() sul dataframe per fornire il *conteggio dei valori non mancanti in ogni colonna* come mostrato nella figura successiva.

```
dfm.describe().show(1)
```

```
+-------+-----------+----------+-----------+----------+-------+
|summary|sepallength|sepalwidth|petallength|petalwidth|variety|
+-------+-----------+----------+-----------+----------+-------+
|  count|        148|       150|        149|       150|    150|
+-------+-----------+----------+-----------+----------+-------+
only showing top 1 row
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per recuperare i record relativi ai valori mancanti nel dataframe di PySpark, si può utilizzare il seguente codice che richiama il metodo **where**() (nella prima figura si mostra l'utilizzo con la colonna 'sepallength', mentre nella seconda figura successiva si mostra l'utilizzo con la colonna 'petallength'):

```
dfm.where(col('sepallength').isNull()).show()
```

```
+-----------+----------+-----------+----------+-------+
|sepallength|sepalwidth|petallength|petalwidth|variety|
+-----------+----------+-----------+----------+-------+
|       null|       3.0|        1.6|       0.2| Setosa|
|       null|       3.4|        1.6|       0.4| Setosa|
+-----------+----------+-----------+----------+-------+
```

```
dfm.where(col('petallength').isNull()).show()
```

```
+-----------+----------+-----------+----------+---------+
|sepallength|sepalwidth|petallength|petalwidth|  variety|
+-----------+----------+-----------+----------+---------+
|        6.1|       3.0|       null|       1.8|Virginica|
+-----------+----------+-----------+----------+---------+
```

La **gestione dei valori mancanti** è una materia complessa della scienza di dati. Diverse, infatti, sono le tecniche utilizzate per gestire i valori mancanti in base al tipo di dati mancanti e all'utilizzo in questione. Si va dai semplici metodi basati sulla logica, agli avanzati metodi statistici come la regressione e il KNN (k-nearest neighbors).

Tuttavia, *indipendentemente dal metodo utilizzato,* per affrontare i dati che presentano valori mancanti bisogna sempre seguire *una di queste due operazioni:*

1. **rimozione di interi record** che hanno valori mancanti;
2. **inserimento di un valore costante** al posto del dato mancante.

Per rimuovere i valori mancanti da una particolare colonna è possibile utilizzare il seguente comando che, dopo aver **eliminato i record in corrispondenza dei valori mancanti**, mostra a schermo il *numero dei nuovi record completi:*

```
#rimuovere i valori mancanti da una particolare colonna
print("Record dopo la rimozione in base ai valori mancanti di 'sepallength':",
      dfm.select('sepallength').dropna().count())
```

```
Record dopo la rimozione in base ai valori mancanti di 'sepallength': 148
```

Nel caso in oggetto, il numero dei record sarà pari a 147 proprio perché sono stati cancellati valori in 3 righe diverse distribuiti su 2 colonne.

Per **sostituire i valori mancanti *nelle colonne*** del dataframe di PySpark con un *valore numerico costante* è necessario sostituire i valori mancanti in entrambe le colonne (in cui mancano i valori) con un valore numerico costante (es. 1) attraverso il seguente codice:

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
y = dfm.select('sepallength','petallength').fillna(1)
```

Successivamente, è possibile contare i valori mancanti nel nuovo dataframe y appena creato che non avrà valori mancanti come dimostrato nella figura successiva.

```
y.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in y.columns]).show()
```

```
+-----------+-----------+
|sepallength|petallength|
+-----------+-----------+
|          0|          0|
+-----------+-----------+
```

È possibile anche **sostituire tutti i valori mancanti nel dataframe** di PySpark *con un valore numerico costante* (es. 1) con la seguente istruzione:

```
z = dfm.fillna(1)
```

Per verificare, poi, la riuscita di tale sostituzione è possibile richiamare la seguente istruzione per verificare il numero di valori mancanti per tutte le colonne del dataframe.

```
z.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in dfm.columns]).show()
```

```
+-----------+----------+-----------+----------+-------+
|sepallength|sepalwidth|petallength|petalwidth|variety|
+-----------+----------+-----------+----------+-------+
|          0|         0|          0|         0|      0|
+-----------+----------+-----------+----------+-------+
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Riproducibilità nell'analisi dei Big Data

La **riproducibilità computazionale** costituisce una grande sfida affrontata oggi in tutte le aziende come anche nel mondo accademico. In particolare, ciò è vero nei lavori di scienza dei dati in cui la maggior parte dei ***dati***, dei ***dataset completi*** e del ***flusso di lavoro*** non è completamente accessibile.

Oggi la maggior parte degli articoli tecnici e di ricerca su un particolare argomento termina con:

- *l'approccio utilizzato sui dati di esempio;*
- *una breve citazione della metodologia utilizzata;*
- *un approccio teorico a una soluzione.*

La maggior parte di questi lavori, infatti, ***non contiene calcoli dettagliati e metodi graduali***.

Ciò rappresenta una quantità molto limitata di informazioni che non consente a chi legge di riprodurre lo stesso lavoro.

Obiettivo della programmazione riproducibile è, quindi, la ***facilità di riproduzione del codice.*** *Jupyter notebook* è stato *perfezionato in tale senso* e, ora, è possibile includere elementi di testo per commentare in dettaglio, migliorando così il processo di riproduzione. Ed è proprio per questo motivo che Jupyter notebook sta guadagnando terreno nella scienza dei dati e nella comunità dei ricercatori. Jupyter notebook è stato sviluppato, infatti, con l'intento di essere un ***software open source con standard e servizi open per l'elaborazione interattive*** di decine di linguaggi di programmazione, tra cui *Python, Spark* e *R.*

La **mancata riproducibilità computazionale** è, quindi, dovuta alla **mancanza di documentazione che si ripercuote tutti i livelli**, dalla *comprensione del metodo* al *codice.* Jupyter è uno dei migliori strumenti per migliorare la riproducibilità computazionale e riutilizzare il codice sviluppato. Esso, infatti, permette di includere non solo la spiegazione di ciò che fa ogni riga o frammento di codice, ma anche la descrizione e la visualizzazione dei dati.

Per iniziare, è possibile individuare alcuni *suggerimenti* per ottenere la ***riproducibilità utilizzando Jupyter notebook in Python***:

1. fornire un'introduzione dettagliata al problema;
2. documentare l'approccio e il flusso di lavoro;
3. spiegare il canale dati;
4. descrivere le dipendenze;
5. utilizzare il controllo della versione del codice sorgente;
6. rendere modulare il processo.

Analizziamo di seguito ciascuno dei sei suggerimenti appena elencati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. **Introduzione al problema**

Uno dei grandi vantaggi dell'utilizzo di Jupyter Notebook è che consente di creare un **flusso di lavoro** inserendo commenti di testo accanto al codice. È utile iniziare con una ***buona introduzione al problema identificato***, a cui dovrebbe seguire un **riepilogo** per indicare il nucleo della questione.

Tale sintesi deve includere anche una ***definizione del problema*** identificato dal punto di vista della *scienza dei dati* e concludersi con il perché è necessario effettuare quell'operazione.

2. **Documentare l'approccio e il flusso di lavoro**

Nel lavoro computazionale della *scienza dei dati,* ci possono essere un sacco di operazioni ripetute. Ad esempio, le indagini effettuate, il tipo di algoritmi utilizzati, le modifiche parametriche.

Documentare il metodo e i flussi di lavoro aiuta a ***definire un processo.*** Inoltre, aggiungere *commenti al codice* durante lo sviluppo è doveroso e deve essere una prassi continua da non eseguire solo al termine dopo aver ottenuto i risultati. È molto facile, infatti, dimenticare i dettagli durante lo sviluppo del codice.

La ***manutenzione di una buona documentazione*** in Jupyter Notebook offre, poi, i seguenti **vantaggi**:

- tracciabilità dello sforzo di sviluppo;
- codice auto esplicativo con commenti per ogni processo;
- una migliore comprensione del flusso di lavoro del codice e dei risultati di ogni fase;
- prevenzione del lavoro ripetuto grazie alla facilità con cui è possibile trovare dei precedenti frammenti di codice che svolgono compiti specifici;
- prevenzione della duplicazione del lavoro grazie alla comprensione dell'uso ripetuto del codice;
- facilità di trasferimento delle conoscenze.

3. **Spiegare il canale dati**

I ***dati*** per identificare e quantificare il problema possono essere ***generati da molteplici fonti di dati*** (es. database, vecchi sistemi, fonti di dati in tempo reale, ecc.). Il *data scientist* lavora a stretto contatto con il team del cliente che si occupa della gestione dei dati per estrarre e raccogliere le **informazioni necessarie** e inserirle negli **strumenti analitici per ulteriori analisi**. Il *data scientist,* inoltre, crea un robusto canale dati per acquisire tali informazioni.

È molto importante documentare in dettaglio le **fonti di dati** per mantenere un ***dizionario dei dati*** che spieghi le variabili considerate, incluso il motivo per cui sono state considerate, la loro forma (strutturate o non strutturate) e il tipo di dati. Per esempio, potrebbe esserci una serie temporale a più variabili o dati che devono essere pre-elaborati e generati da risorse grezze come immagini, testi, registrazioni audio.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. **Spiegare le dipendenze**

Le dipendenze sono i pacchetti e le librerie disponibili in uno strumento. Ad esempio, è possibile utilizzare:

- *Matplotlib*, per la visualizzazione dei grafici;
- *TensorFlow*, un'API per la modellazione del deep learning.

Le **dipendenze possono includere anche specifiche hardware e software** necessarie per un'analisi. È possibile gestire dall'inizio le dipendenze in modo esplicito utilizzando strumenti come Conda ed elencando tutte le dipendenze pertinenti incluse le ***versioni dei pacchetti/librerie.***

5. **Uso del controllo della versione del codice sorgente**

Il controllo della versione del codice sorgente è un aspetto importante in qualunque tipo di attività computazionale che coinvolge il codice. Quando il codice è in **fase di sviluppo**, emergono ***bug ed errori.***

Se sono disponibili versioni precedenti del codice, sarà possibile determinare il momento in cui è stato identificato il bug, quando è stato risolto e l'entità dello sforzo richiesto, il tutto possibile *grazie al controllo della versione*. Per motivi di scalabilità o prestazioni, potrebbe anche essere **necessario tornare alle versioni precedenti.**

6. **Modularizzazione del processo**

Evitare di duplicare il codice è una prassi efficace per la gestione delle attività ripetitive, la manutenzione del codice e il debug. Si propone un esempio per chiarire la modularizzazione del codice. Si supponga di eseguire una serie di procedure di manipolazione dei dati e di sviluppare il codice che completa un'attività. Ipotizzando di dover usare lo stesso codice in una sezione successiva del programma bisognerebbe aggiungere, copiare o ripetere gli stessi passi (***compito ripetitivo***).

I dati di input e i nomi delle variabili possono cambiare nelle diverse sezioni (ripetute) del codice, ma per gestire tutto questo basterebbe:

- scrivere i passaggi precedenti sotto forma di ***funzione per un dataset o una variabile;***
- salvare tutte queste funzioni in un ***modulo separato*** che può essere considerato come una sorta di file di funzioni (libreria Python).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

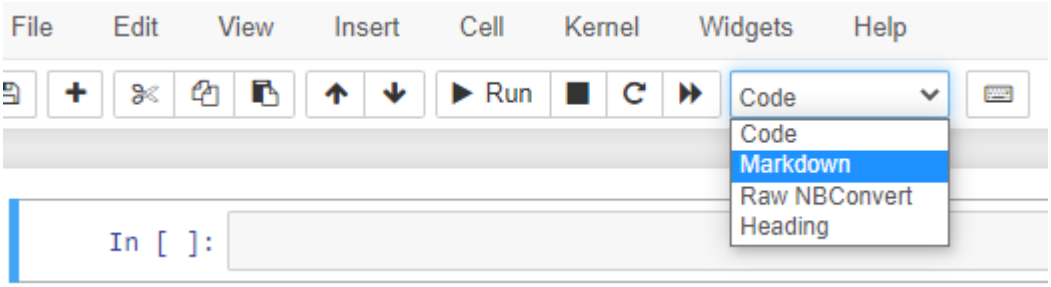
4. Raccolta dei dati in modo riproducibile

Una volta definito il problema, il primo passo di un'attività di analisi è la ***raccolta dei dati.***

Essendo diverse le **fonti** da cui i dati possono essere estratti e i **modi** in cui tali dati possono essere caricati nel modello, si introduce una *modalità di scrittura* ***markdown*** *e* ***codice*** in Jupyter notebook.

Il testo può essere aggiunto in Jupyter notebook utilizzando le **celle markdown.** Tali blocchi di testo possono essere scritti in grassetto, in corsivo o senza particolari formati come in un qualunque editor di testo. Per cambiare in markdown il tipo di cella si può usare il menu **Cell.**

Si esaminano, ora, i modi in cui è possibile utilizzare in Jupyter varie funzionalità nelle celle markdown e nelle celle di codice. Selezionare per una particolare cella l'**opzione Markdown in Jupyter** come illustrato nella figura successiva.

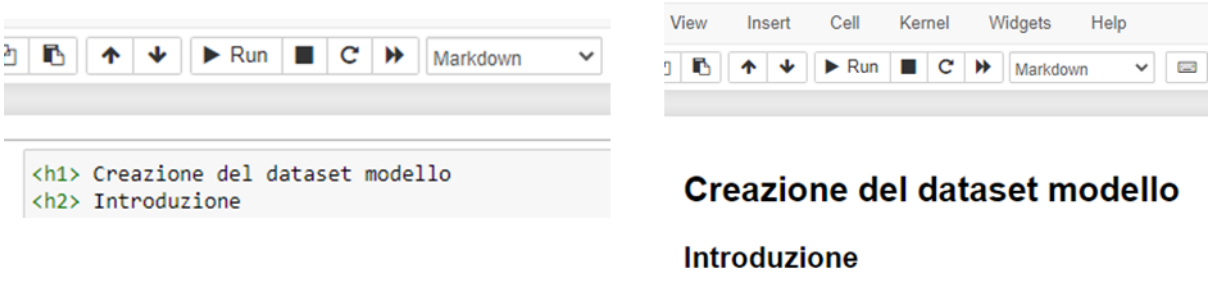


La sintassi per raggiungere un titolo in una cella markdown è simile a quella di HTML.

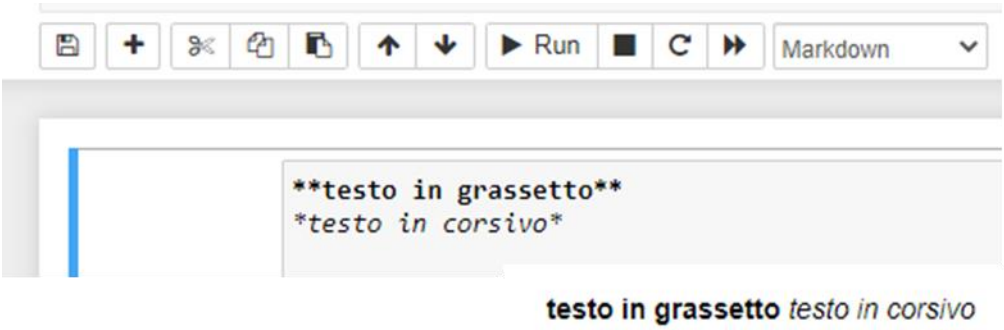
Esistono **due tipi di intestazioni in Jupyter notebook:** si usano, infatti, i tag ***<h1>*** e ***<h2>.***

Per visualizzare la versione finale (a destra) del contenuto della cella markdown (a sinistra nella figura seguente) è necessario premere **Run.**

Il testo può essere aggiunto direttamente così com'è. Per quanto riguarda invece il **grassetto** e il *corsivo*, invece, il grassetto si ottiene con i **doppi asterischi **** (all'inizio e alla fine), mentre il corsivo si ottiene con i *singoli asterischi ** (all'inizio e alla fine).



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Fatte queste premesse, è molto importante *iniziare il flusso* fornendo una breve **introduzione al problema per comprendere l'obiettivo del progetto.** La **definizione del problema** è una *sintesi dell'esposizione del problema* e include il modo in cui il problema può essere risolto utilizzando un algoritmo di *data science.* La figura seguente mostra un esempio di breve introduzione al problema delle sottoscrizioni.



All'interno del codice, inoltre, è necessario documentare correttamente la fonte dei dati per **comprendere la riproducibilità.** Un esempio di come è possibile aggiungere la fonte di dati è la seguente:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
# Legge il dataset di input come "df" usando la funzione read_csv di pandas
df = pd.read_csv('bank.csv',sep=';')

# visualizza le prime 5 righe del dataset usando la funzione head
df.head(5)
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	30	unemployed	married	primary	no	1787	no	no	cellular	19	oct	79	1	-1	0	unknown	no
1	33	services	married	secondary	no	4789	yes	yes	cellular	11	may	220	1	339	4	failure	no
2	35	management	single	tertiary	no	1350	yes	no	cellular	16	apr	185	1	330	1	failure	no
3	30	management	married	tertiary	no	1476	yes	yes	unknown	3	jun	199	4	-1	0	unknown	no
4	59	blue-collar	married	secondary	no	0	yes	no	unknown	5	may	226	1	-1	0	unknown	no

Ai fini della riproducibilità è necessario mantenere, inoltre, un **dizionario dei dati** per comprendere i dati a livello di attributo (**Data Dictonary**). Come nell'esempio seguente, è preferibile includere la **definizione dell'attributo con il suo tipo di dati.**

Dizionario dei dati

Fornisce informazioni dettagliate a livello di attributo:

1 - age (numerico);

2 - job: tipo di occupazione (categoriale: "admin", "blue-collar", "entrepreneur", "housemaid", "management", "retired", "self-employed", "services", "student", "technician", "unemployed", "unknown")

3 - marital: stato civile (categoriale: "divorced", "married", "single", "unkown", nota: "divorced" può indicare divorziato o vedovo);

4 - education (categoriale: "basic.4y", "basic.6y", "basic.9y", "high school", "illeterate", "professional.course", "university.degree", "unknown");

5 - default: ha un credito in mora? (categoriale: "no", "yes");

6 - housing: ha un mutuo sulla casa? (categoriale: "no", "yes");

A questa dovrebbe seguire la sezione **Data Understanding** (*comprensione dei dati*). Nel caso di esempio, nella figura successiva, vengono forniti i **dati complessivi** che comprendono tutte le righe e le colonne considerate.

Comprendere i dati

Per comprendere i dati a livello di attributo possiamo usare funzioni come info e describe; pandas_profiling è una libreria che fornisce molte informazioni descrittive in una funzione che permette di estrarre le seguenti informazioni: a livello di dataset:

1. numero di variabili;
2. numero di osservazioni;
3. totale mancanti (in %);
4. dimensioni totali in memoria;
5. dimensione media dei record in memoria;
6. matrice di correlazione;
7. dati di esempio.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ed è proprio per comprendere i dati a livello di attributo che torna molto utile l'utilizzo della **libreria pandas_profiling.** Tale libreria fornisce molte *informazioni descrittive* in **un'unica funzione** da cui è possibile estrarre diverse informazioni.

Prima di poter utilizzare la libreria, però, è necessario installarla nel sistema in modo tale da essere utilizzabile con Jupyter notebook. Avviare come Amministratore il file **cmd.exe** (click destro mouse ed «*Esegui come Amministratore*) solitamente nella cartella *C:\Windows\System32.* Digitare il seguente comando:

python -m pip install pandas-profiling

Avviare l'*Anaconda Prompt* digitando il seguente comando:

C:\ProgramData\Anaconda3\Scripts\activate.bat C:\ProgramData\Anaconda3

Digitare il seguente comando:

conda install –c anaconda pandas-profiling

La **libreria pandas_profiling** contiene un **bug** che è necessario risolvere manualmente per poter sfruttare le *funzionalità della libreria stessa.* In particolare, **l'errore** è dovuto ad un ***parametro*** che non è più utilizzato nei codici. Nella figura seguente si mostra l'errore che viene segnalato.

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-15-02182f49e9ae> in <module>
      4
      5 print(df)
----> 6 pandas_profiling.ProfileReport(df)

C:\ProgramData\Anaconda3\lib\site-packages\pandas_profiling\__init__.py in __init__(self, df, **kwargs)
     64         sample = kwargs.get('sample', df.head())
     65
---> 66         description_set = describe(df, **kwargs)
     67
     68         self.html = to_html(sample,

C:\ProgramData\Anaconda3\lib\site-packages\pandas_profiling\describe.py in describe(df, bins, check_correlation, correlation_th
reshold, correlation_overrides, check_recoded, pool_size, **kwargs)
    390             if name not in names:
    391                 names.append(name)
--> 392     variable_stats = pd.concat(ldesc, join_axes=pd.Index([names]), axis=1)
    393     variable_stats.columns.names = df.columns.names
    394

TypeError: concat() got an unexpected keyword argument 'join_axes'
```

La risoluzione di tale bug prevede la **modifica del codice del file**

C:\ProgramData\Anaconda3\lib\site-packages\pandas_profiling\describe.py

La nuova versione del file a partire dalla riga 385 (e per le righe mostrate) dovrà essere modificata nel seguente modo:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    # Convert ldesc to a DataFrame
    names = []
    ldesc_indexes = sorted([x.index for x in ldesc.values()], key=len)
    for idxnames in ldesc_indexes:
        for name in idxnames:
            if name not in names:
                names.append(name)
    variable_stats = pd.DataFrame (ldesc)
    #variable_stats = pd.concat(ldesc, keys=pd.Index([names]), axis=1)
    #variable_stats.columns.names = df.columns.names
```

Attraverso il **metodo ProfileReport**() a cui viene passato il dataframe pandas, è possibile ricavare diverse e importanti informazioni. La prima parte del risultato denominata **Overview** riporta le informazioni riportate nella figura successiva.

```
import pandas_profiling

pandas_profiling.ProfileReport(df)
```

Overview

Dataset info

Number of variables	17
Number of observations	4521
Total Missing (%)	0.0%
Total size in memory	600.6 KiB
Average record size in memory	136.0 B

Variables types

Numeric	7
Categorical	10
Boolean	0
Date	0
Text (Unique)	0
Rejected	0
Unsupported	0

Warnings

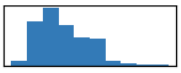
- balance has 357 / 7.9% zeros Zeros
- previous has 3705 / 82.0% zeros Zeros

A livello di attributo/variabile (sezione **Variables**), cioè per ciascuna colonna, le specifiche fornite sono: conteggio netto, valori unici (%), valori mancanti (% e numero), infiniti (% e n), istogramma per la distribuzione, valori estremi come è possibile osservare dalla figura seguente.

Variables

age
Numeric

Distinct count	67	Mean	41.17
Unique (%)	1.5%	Minimum	19
Missing (%)	0.0%	Maximum	87
Missing (n)	0	Zeros (%)	0.0%
Infinite (%)	0.0%		
Infinite (n)	0		



Toggle details

job
Categorical

Distinct count	12
Unique (%)	0.3%
Missing (%)	0.0%

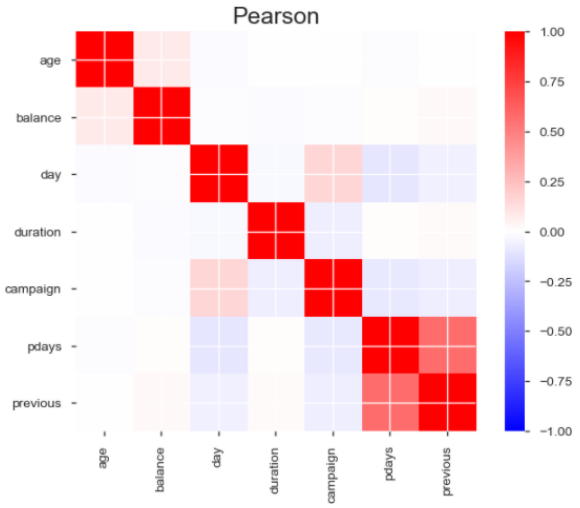
management 969
blue-collar 946

Infine, nel rapporto realizzato con il metodo **ProfileReport**() vengono fornite anche le **tabelle di correlazione** (si veda figura successiva) e vengono visualizzate le *prime righe del dataframe.*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Correlations



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Riproducibilità del codice

Sempre in ottica di riproducibilità, è importante scrivere il codice seguendo una serie di prassi e standard spiegando in modo descrittivo e graduale il **flusso di lavoro del processo.**

Ovviamente, ciò è universalmente applicabile con tutti gli strumenti di programmazione utilizzabili, non solo con Jupyter.

Ai fini dell'installazione si dovrebbe mantenere un ***frammento di codice dedicato all'installazione dei pacchetti e delle librerie necessarie.*** Le seguenti prassi agevolano, infatti, la riproducibilità del codice:

- includere le **versioni usate per le librerie e i pacchetti;**
- scaricare le **versioni originali** di pacchetti/librerie usati.

Per quanto riguarda il codice, invece, esso deve essere **leggibile e opportunamente commentato.** I commenti sono necessari per motivare, ad esempio, perché è stato utilizzato uno specifico metodo.

Anche la ***scelta dei nomi degli oggetti è fondamentale per la riproducibilità e la comprensione.*** Ad esempio, è opportuno indicare e diversificare i dati in *grezzi*, dati *del modello*, dati *pre-elaborati*, dati *di analisi.* Durante lo sviluppo del codice si progettano delle fasi per raggiungere i risultati finali. **Ogni fase può far parte di un processo**. Per esempio, leggere i dati, capire i dati, eseguire varie trasformazioni o costruire un modello.

Ciascuna di queste fasi deve essere ben distinta per diverse ragioni tra cui:

- la leggibilità del codice per il modo in cui ogni fase è stata realizzata;
- il modo in cui viene generato il risultato di ogni fase.

Sempre per la riproducibilità del codice, anche la **documentazione del flusso di lavoro** gioca il suo ruolo. Essa include informazioni su:

- requisiti e progettazione;
- documentazione sul prodotto;
- informazioni sulla metodologia;
- guide all'installazione;
- manuali d'uso del software;
- requisiti hardware e software;
- gestione della risoluzione dei problemi;
- documenti di test.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La documentazione del flusso di lavoro aiuta durante le **fasi di implementazione e integrazione in un ambiente cliente/utente.** Ad alto livello la documentazione di un progetto di data science può essere suddivisa in due segmenti:

1. **documentazione del prodotto** che fornisce informazioni sul modo in cui ogni *funzionalità viene utilizzata nell'interfaccia utente* e la sua applicazione. Tale documentazione può essere ulteriormente segmentata in guide all'installazione, progettazione software e manuale utente, documenti di prova, risoluzione del problema;
2. **documentazione metodologica** che fornisce informazioni su algoritmi utilizzati, metodi e approccio alla soluzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Marin I., Shukla A. Sarang VK. (2019). L'analisi dei Big Data con Python. Tecniche nuove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).